

Group 2: Installing and Running SWI-Prolog

By Emily Herman, Megan Ruiz, Erik Ellams, Nickolas Diaz

Using Prolog Effectively

- Prolog is a declarative language, meaning describing what the desired outcome is, rather than explicitly listing the steps to achieve it.
- A building in prolog involves building facts and rules that define a problem and the relationships between objects, this is called a knowledge base.
- From a knowledge base queries/questions can be made, and the prolog interpreter will automatically find the solutions.

Writing Facts

- Facts are used to state things that are unconditionally true of some situation of interest.
- Structure: a **relationship/property name** followed by one or more arguments, then terminated by a **period "."**

Prolog Syntax	English Translation
<code>pet(cat).</code>	A cat is a pet.
<code>parent(john, mary).</code>	John is the parent of Mary.
<code>age(mary, 25).</code>	Mary's age is 25.
<code>sunny.</code>	It is sunny.

Writing Rules

- A rule is a predicate expression that uses logical implication to describe a relationship among facts.
- Structure: `left_hand_side :- right_hand_side .`
- This sentence is interpreted as: `left_hand_side` if `right_hand_side`.

Prolog Syntax	English Translation
<code>friends(X,Y) :- likes(X,Y),likes(Y,X).</code>	X and Y are friends if they like each other.
<code>friends(X, Y) :- likes(X, Food), likes(Y, Food), X \= Y.</code>	X and Y are friends if they like the same food and X and Y are not the same person.

Symbols

- Prolog expressions are comprised of the following truth-functional symbols

Prolog Syntax	English Translation
,	Or
;	And
:-	If
not	Not
<code>pet(Animal, Person) :- cat(Animal), dog(Animal); not(hates(Person)).</code>	A pet is an animal that is dog or cat and is not hated by a person.

Debugging queries

Debugging Commands	Purpose
<code>trace.</code>	Run for any subsequent query to be shown step-by-step
<code>notrace.</code>	Turns off the tracer
<code>spy(PredicateName).</code>	Sets a "breakpoint" on a specific predicate.
<code>nospy(PredicateName).</code>	Removes the spy point.
<code>nodebug.</code>	Switches off all debugging and spy points.
<code>listing(Predicate).</code>	Displays the source code for a specific predicate.
<code>break.</code>	Pauses the current execution and lets you run other queries in a "sub-shell."
<code>statistics.</code>	Shows memory and stack usage (vital for finding infinite loops).

Common errors beginners make (Part 1 of 2)

trace Commands	Purpose
c (Creep)	Move to the very next step.
s (Skip)	Run the current goal without showing its internal details.
l (Leap)	Fast-forward to the next spy point.
a (Abort)	Stop the execution and return to the normal prompt.

- When using trace many beginners forget that the debugger waits for user input at every step.
- Beginners often think the program has frozen when it is just waiting for one of these keys.

Common errors beginners make (Part 2 of 2)

Common Error	Result	The Fix
Missing the period (.)	Interpreter hangs/stays silent.	Always end queries with a period.
Wrong Case	"House" is a variable, "house" is a person.	Lowercase for facts, Uppercase for variables.
"=" vs "is"	<code>X = 1 + 1</code> stays as a string.	Use "is" for mathematical calculations.
Infinite Recursion	Example Code: <pre>ancestor(X, Y) :- ancestor(X, Z), parent(Z, Y).</pre> %Error: Calls ancestor before parent Causing Stack Overflow / Freeze.	Ensure your "base case" comes before the recursion. Move the 'known fact' to the left <pre>ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).</pre>

More Complex Examples (Family Tree)

- A family tree is a common Prolog example because relationships can be represented with facts and rules.
- Facts store basic relationships between people.
- Rules allow Prolog to infer new relationships from existing ones.

FACTS

Prolog Syntax	English Translation
<code>parent(john, mary).</code>	John is the parent of Mary.
<code>parent(mary, sam).</code>	Mary is the parent of Sam.
<code>parent(john, alex).</code>	John is the parent of Alex.
<code>parent(alex, lily).</code>	Alex is the parent of Lily.

RULES

Prolog Syntax	English Translation
<code>grandparent(X,Y) :- parent(X,Z), parent(Z,Y).</code>	X is a grandparent of Y if X is a parent of Z and Z is a parent of Y.
<code>ancestor(X,Y) :- parent(X,Y).</code>	X is an ancestor of Y if X is a parent of Y.
<code>ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).</code>	X is an ancestor of Y if X is a parent of Z and Z is an ancestor of Y

```
?- grandparent(john,  
sam).           true.
```

```
?- ancestor(john,  
lily).          true.
```

More Complex Examples (Pathfinding)

- Prolog can also represent paths in a graph or network.
- Facts represent connections between locations.
- Rules allow Prolog to determine if a path exists between two points.

FACTS

Prolog Syntax	English Translation
<code>connected(a,b).</code>	A is connected to B.
<code>connected(b,c).</code>	B is connected to C.
<code>connected(c,d).</code>	C is connected to D.
<code>connected(a,e).</code>	A is connected to E.

RULES

Prolog Syntax	English Translation
<code>path(X,Y) :- connected(X,Y).</code>	There is a path from X to Y if they are directly connected.
<code>path(X,Y) :- connected(X,Z), connected(Z,Y).</code>	There is a path from X to Y if X connects to Z and Z connects to Y.

`?- path(a,c).` `true.`

`?- path(a,d).` `true.`

How Prolog Actually Solves Queries (Backtracking)

EXAMPLE QUERY: `?: ancestor(john, lily).`

- Prolog searches for solutions by trying rules and facts in order.
- If a rule fails, Prolog automatically goes back and tries another possibility.
- This process is what backtracking is.

STEP 1: Try First Rule

`ancestor(X,Y) :- parent(X,Y).`

CHECK: `parent(john, lily).`

RESULT: `false.`

STEP 2: Backtrack and Try Another Rule

`ancestor(X,Y) :- parent(X,Z), ancestor(Z,Y).`

FIND MATCHING FACT: `parent(john, alex).`

PROLOG MUST SOLVE: `ancestor(alex, lily).`

STEP 3: Continue Searching

CHECK: `parent(alex, lily).`

RESULT: `true.`

FINAL RESULT: `?- ancestor(john, lily). true.`